



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Modeling and Scalability

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria
michele.colajanni@unibo.it

Two scenarios

You have a system
(*brownfield*)



You can measure:

- *workload*
- *performance*
- *availability*

You do not have a system
(*greenfield*)



You have to model



Modeling is the most important activity for human progress
(It represents the core business of any scientist and engineer)



Modeling

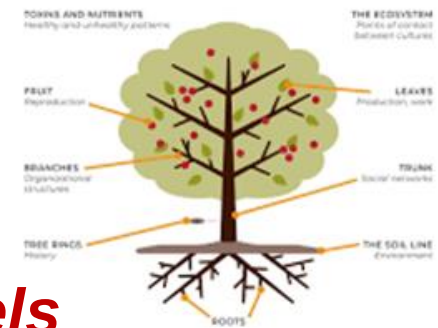
A model is a representation of the reality that is created for a specific purpose. **Real systems that are worth modeling have too many details**, hence modeling requires an abstraction activity, which aims to hide unimportant details in order to focus on essential characteristics for that specific goal



Real system



Models



Abstraction is necessary for design and management

A model is a representation of the reality that is created for a specific purpose. **Real systems have too many details**, hence modeling requires an abstraction activity, which aims to hide unimportant details in order to focus on essential characteristics for that specific goal

Key steps

1. You need a purpose, a goal
2. Only on this basis, you can decide which details of the real system are important and which are unimportant
3. Your model includes only essential characteristics

Easy? NOT AT ALL! Try...



George Box and “useful” models

**“Essentially, all models are wrong,
...but some are useful.”**



- George Box

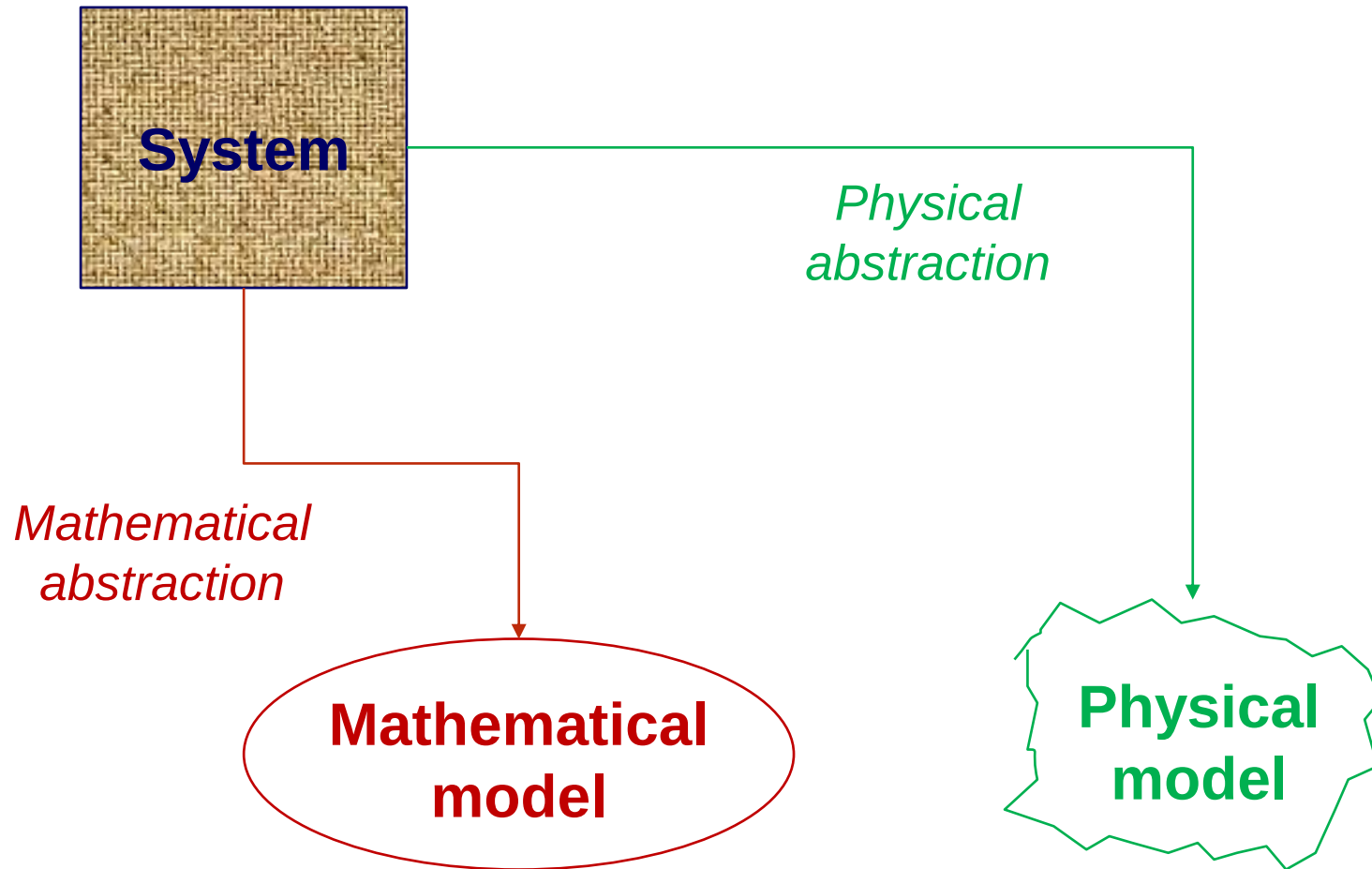
(One of the most influential statisticians of the 20th century and a pioneer in the areas of quality control, time series analysis, design of experiments and Bayesian inference.)

ORACLE

19 Copyright © 2013, Oracle and/or its affiliates. All rights reserved. |



Systems and models: *once upon a time*



Then?

Which is the most important contribution of **Computer Science** (or Computer Engineering or Informatics) to the history of human progress - modeling, design and engineering?

YES, **SOFTWARE**

What does software allow you to do?

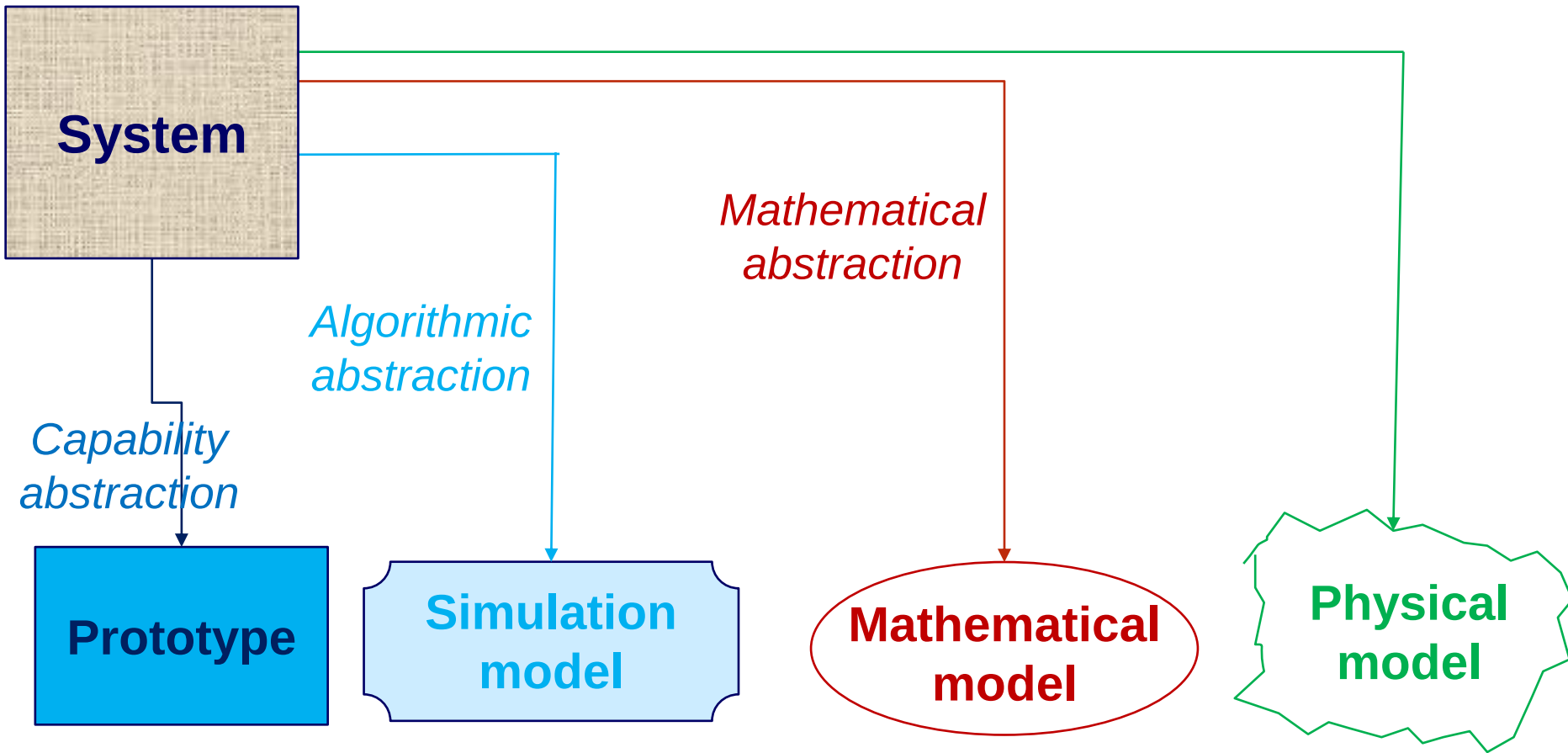


Model representation

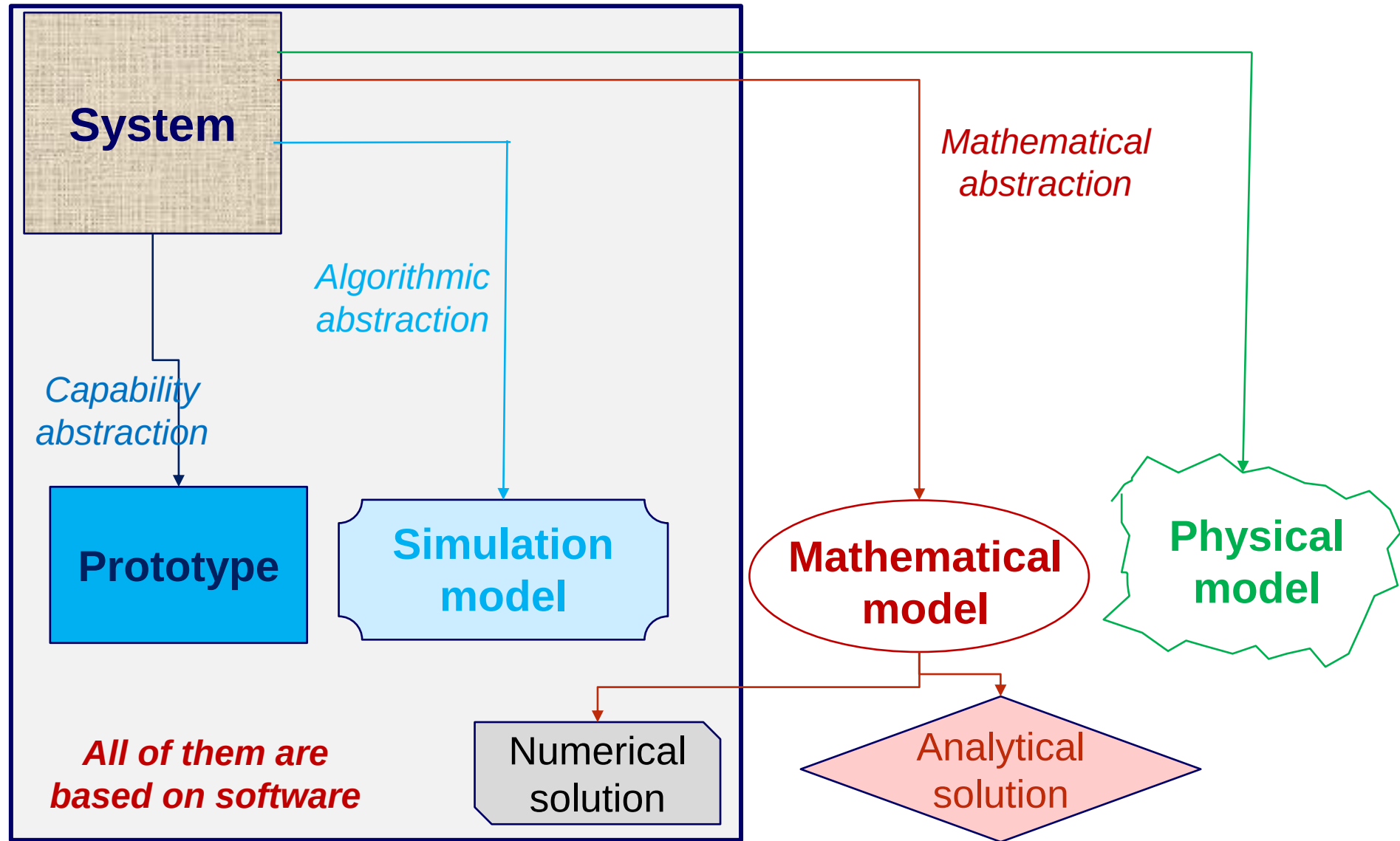
- **Physical models**
- **Mathematical models**
- **Computational models**



Systems and models



Systems and models, and computational domain



Example (1)

As a computer engineer you are requested to (support the) design of the right hardware-software architecture of a new Internet-based service including the comparison of different alternatives

What will you do?



Requirements

- Type of service and sub-services (*functional*)

In terms of (*non-functional requirements*):

- desired **performance**
- desired **reliability**
- desired **scalability**

- Subject to:

- ...
- ...
- ...

SYSTEM
(Service provider)

USER
(Service consumer)



Example 2

As a computer engineer you are requested to verify and eventually improve the reliability and performance of an existing Internet-based service

What will you do?



Lesson learned

There are five factors that influence the quality (in terms of *performance*, *scalability* and *reliability*) of an Internet-based service and, in general, of any application system:

1. **Service description**
2. **Workload** to which the service is or will be subjected
 - **Not existing** → team discussion
 - **Existing** → measurement and team interpretation
3. **Service design and architecture**
4. **Service implementation**
5. **Provisioned platform for execution**
6. **Operational management**

Requirements

Solution



Example 3

As a computer engineer you are requested to check whether the performance of an existing e-commerce service will support an increment of 30-40% of requests during the “golden month” [from Thanksgiving to Christmas]

What will you do?



Definition: “Workload”

“The amount of work to be done”

“The amount and type of service requests expected from the external by some number of users and jobs connected to and interacting with the system in a specified time period”



Importance of workload

Workload characterization is a very important topic for designing and building Internet-based services with scalable, high performance and reliable features

- Designing and/or evaluating a system with the wrong workloads will lead to irrelevant results, which cannot be relied upon
- If the workload is wrong, the results will be wrong too (**GIGO**: *Garbage In – Garbage out*), not only in the mathematical sense, but in the sense that the system:
 - either will be unable to face the real number and type of requests and reliability
 - or will be over-dimensioned



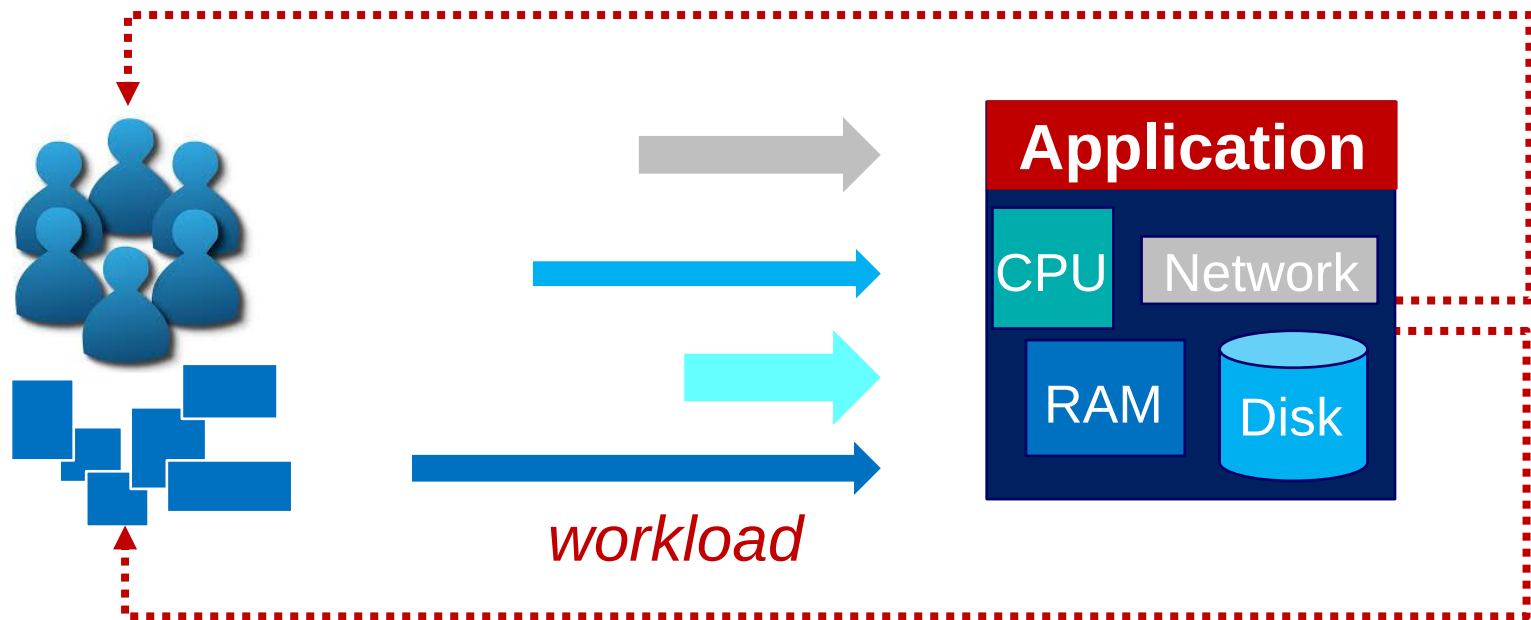
Example

- Is it always true that a sorting algorithm **A** having complexity $O(n \log n)$, is always faster than a sorting algorithm **B** having complexity $O(n^2)$?
- The computational complexity is typically a **worst-case analysis**, which may occur, for example, only for a specific workload
- Different inputs may lead to different performance results. The same algorithm may terminate in linear time if the input is almost sorted to begin with, but may require quadratic time if the input is sorted in the opposite order

Performance $\neq$ Computational complexity



System components and several workload measures



- User requests
- Processing load on servers
- Network traffic
- Amount of data stored
- Amount of read/write to a database
- ...

Don't forget "in a specified time period"



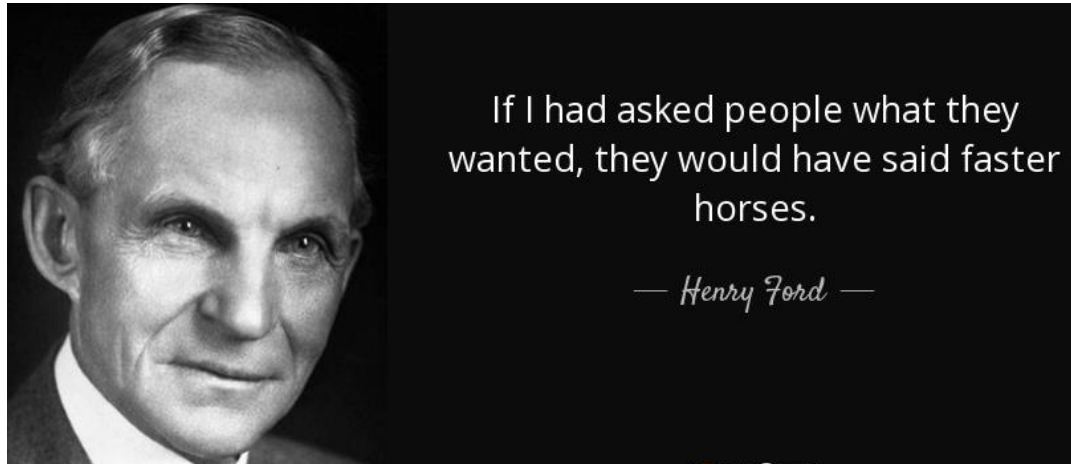
Several goals for modeling

- 1. Functional modeling**
- 2. Systems architecture**
- 3. Business process modeling**
- 4. Architecture modeling**
- 5. Enterprise modeling**
- 6. Social and political modeling**
- 7. ...**



Listening to customers. Yes, but ...

- **Listening too closely to customers is not always the best strategy for innovation** → Customers tell you what they want today, but that is no promise they will buy when the time comes
- *Customers can easily describe a problem they are having (in that case, wanting to get somewhere **faster** now), but not the **best solution***



- **Vice versa, the world is full of solutions that are looking for a problem and/or do not address a real issue precepted by customers**

Why the *workload characterization* is so important

- 1) **Architecture design and provisioning *on-premise***
 - Vital for infrastructure provisioning so to guarantee performance and reliability (and acceptable user experience)
 - Important for costs
- 2) **Architecture design and provisioning *on cloud***
 - Vital for costs
 - For performance and reliability, it is important to design and implement an elastic application
- 3) **System testing**
- 4) **Comparison of alternatives: different systems or designs**
- 5) **Expected system evolution → *Capacity planning***



Scalability

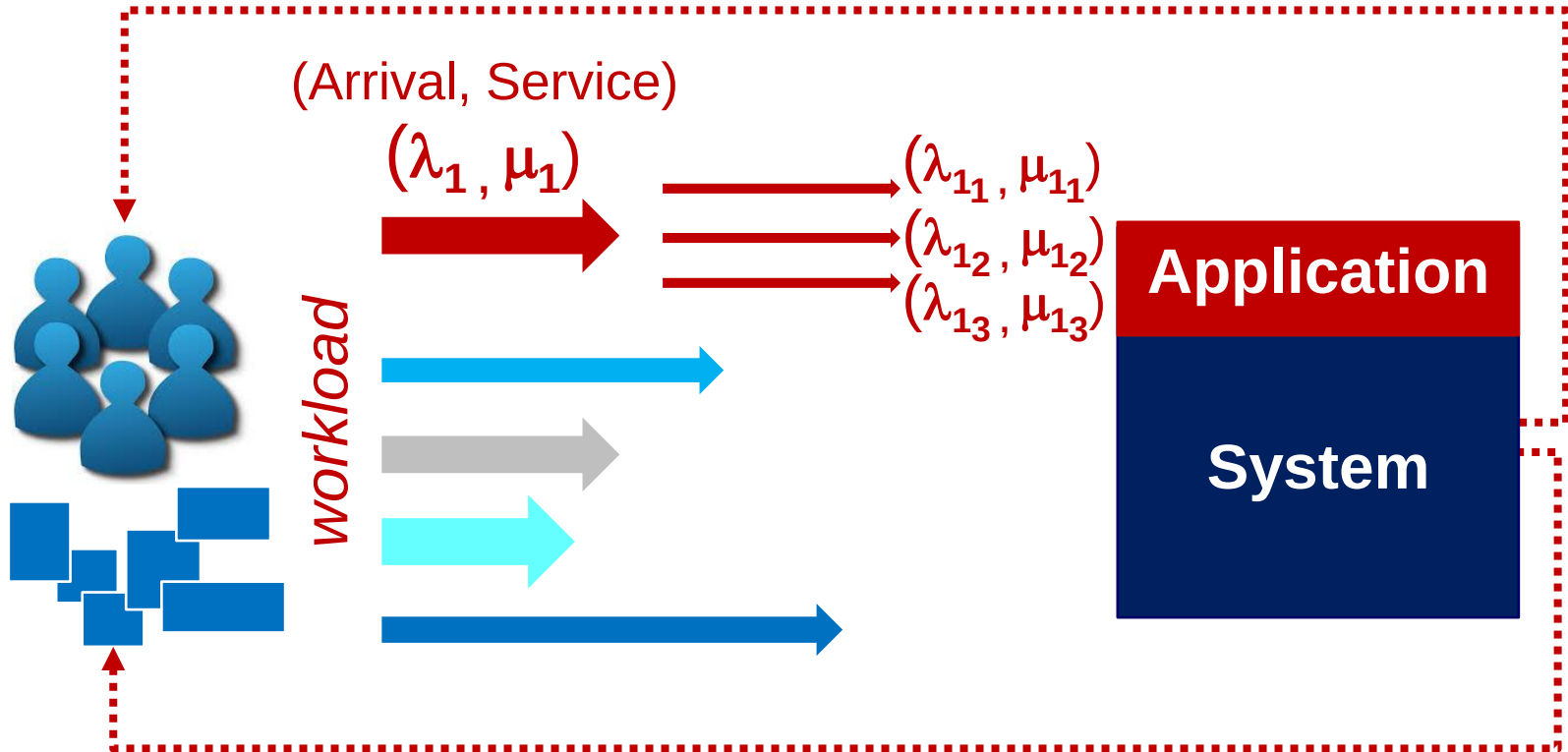


Performance has to do with *System capacity* ...

- Ideal system capacity is the **maximum possible output** of a system that is measured in units of output **per observation period**
- Real system capacity considers that in the observation period the system might run slower than it ideally does
This may be due to:
 - small stops that do not qualify as downtime
 - increase of internal tasks
 - short-range inefficiencies
 - slower input rates
 - ...

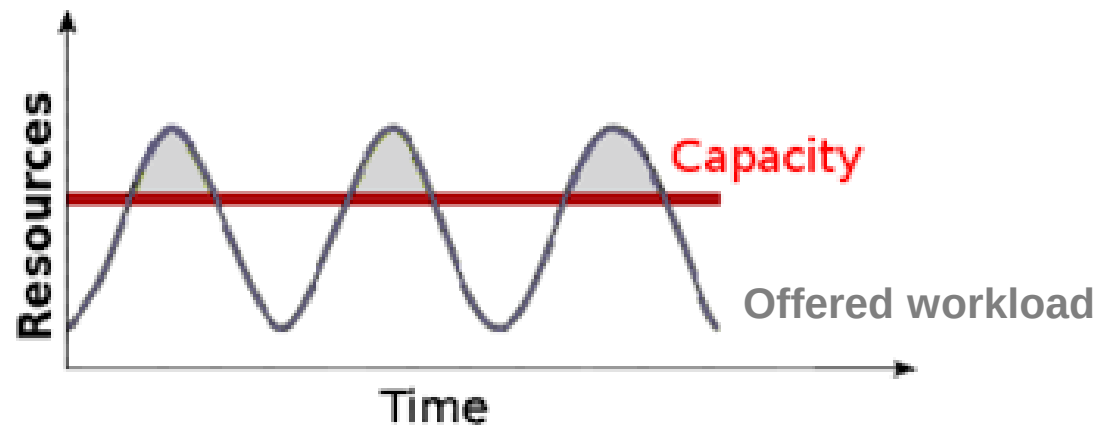
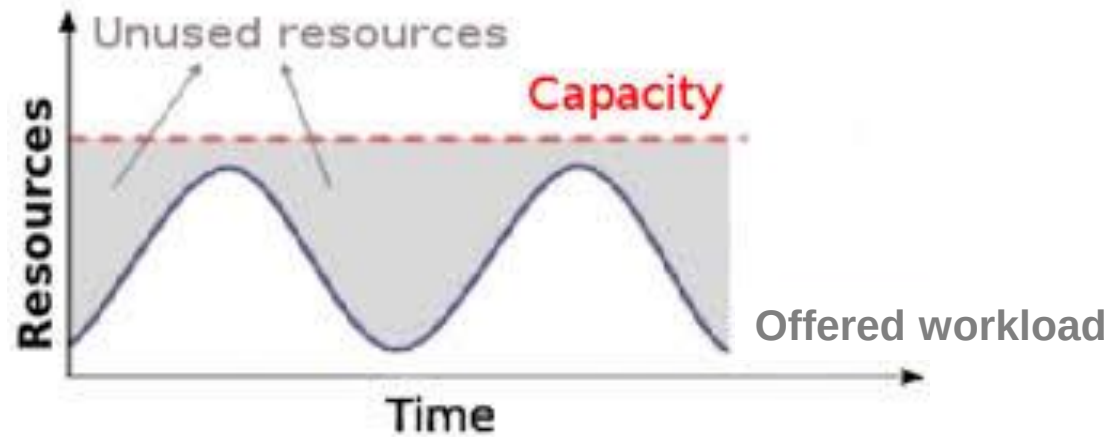


... and with the *offered workload*



A workload model should represent one or more profiles in which the evaluation of the operating environment can be undertaken

Workload and System capacity



Dozen of measures where two are the most important

- Response time (*user vision*)
- System response time (*system vision*)
- System throughput (*system vision*)
- Waiting time
- Number of processes in queue
- Processing time
- Resource (e.g., CPU, disk) utilization
- MIPS/MFLOPS
- Speedup
- Network latency
- Cache hit rate
- Capacity utilization rate
- ...



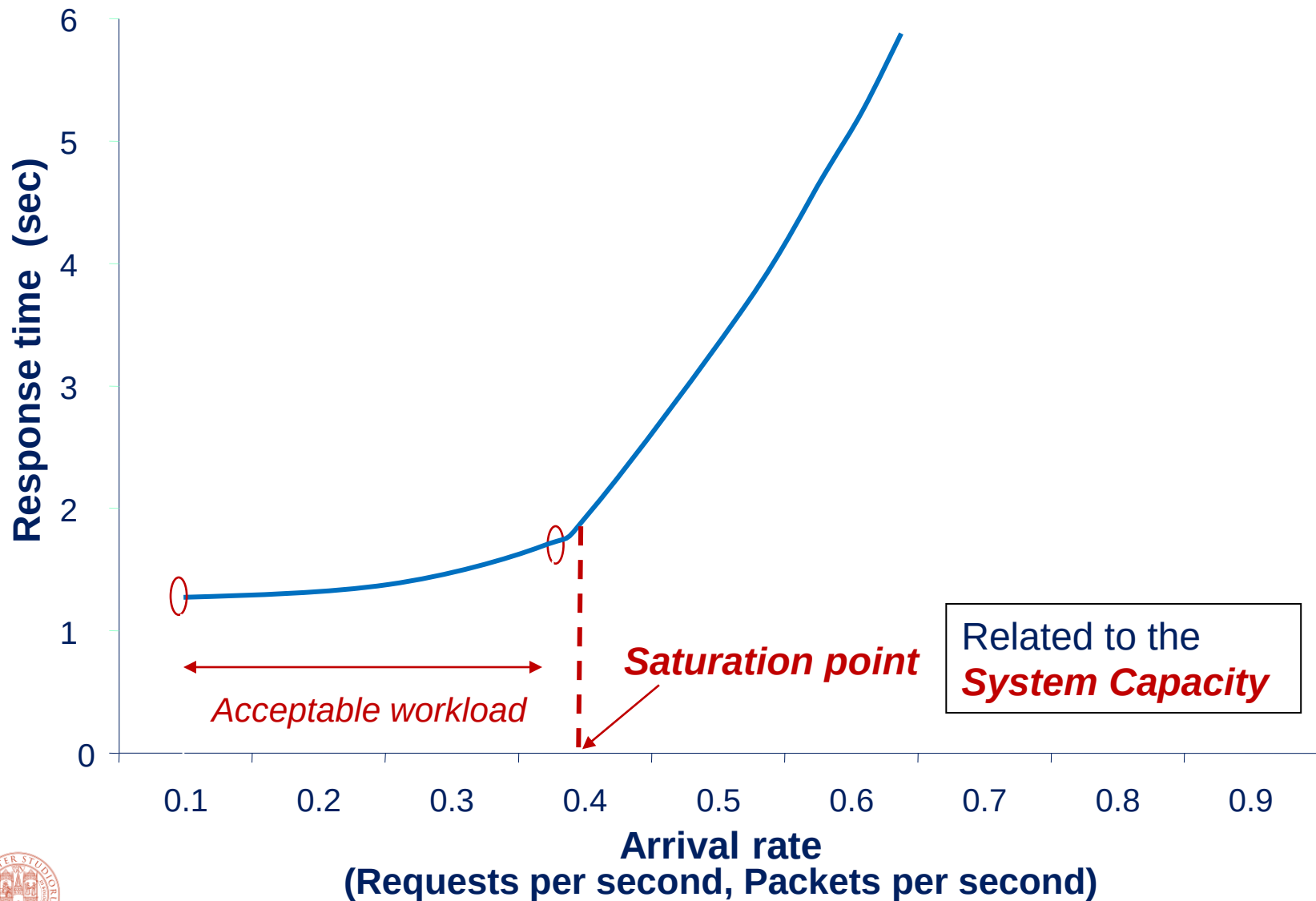
Response time

System response time = Service time + Waiting time

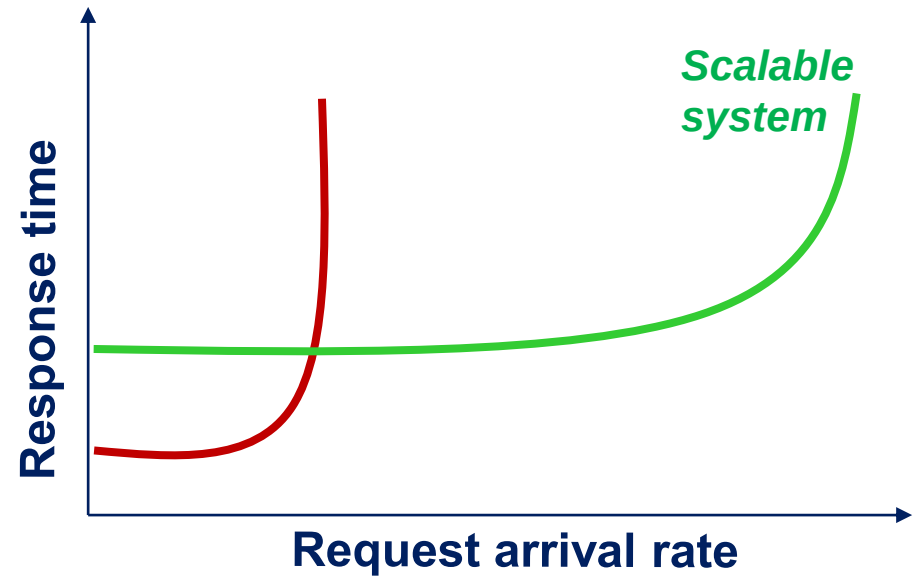
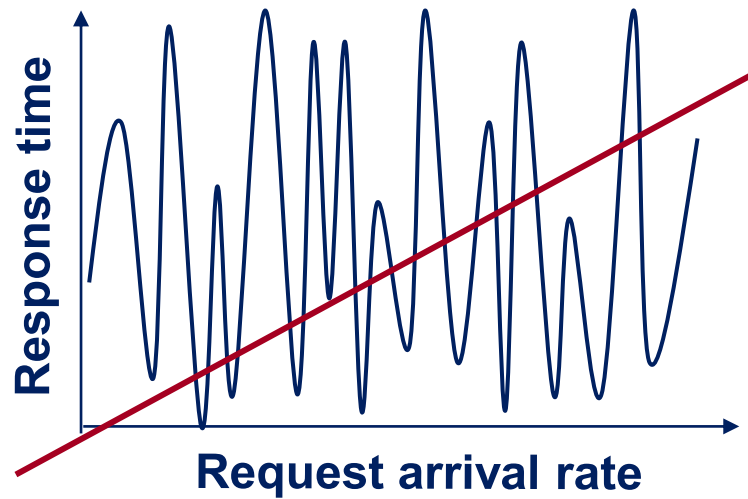
User response time = Network time +
Service time + Waiting time



System response time as a function of the arrivals



System behavior: invalid and valid



System throughput

The **throughput** is the amount of work that a system can process in a given time period

- It is a pure number
- It cannot be higher than
 - the offered workload
 - the system capacity

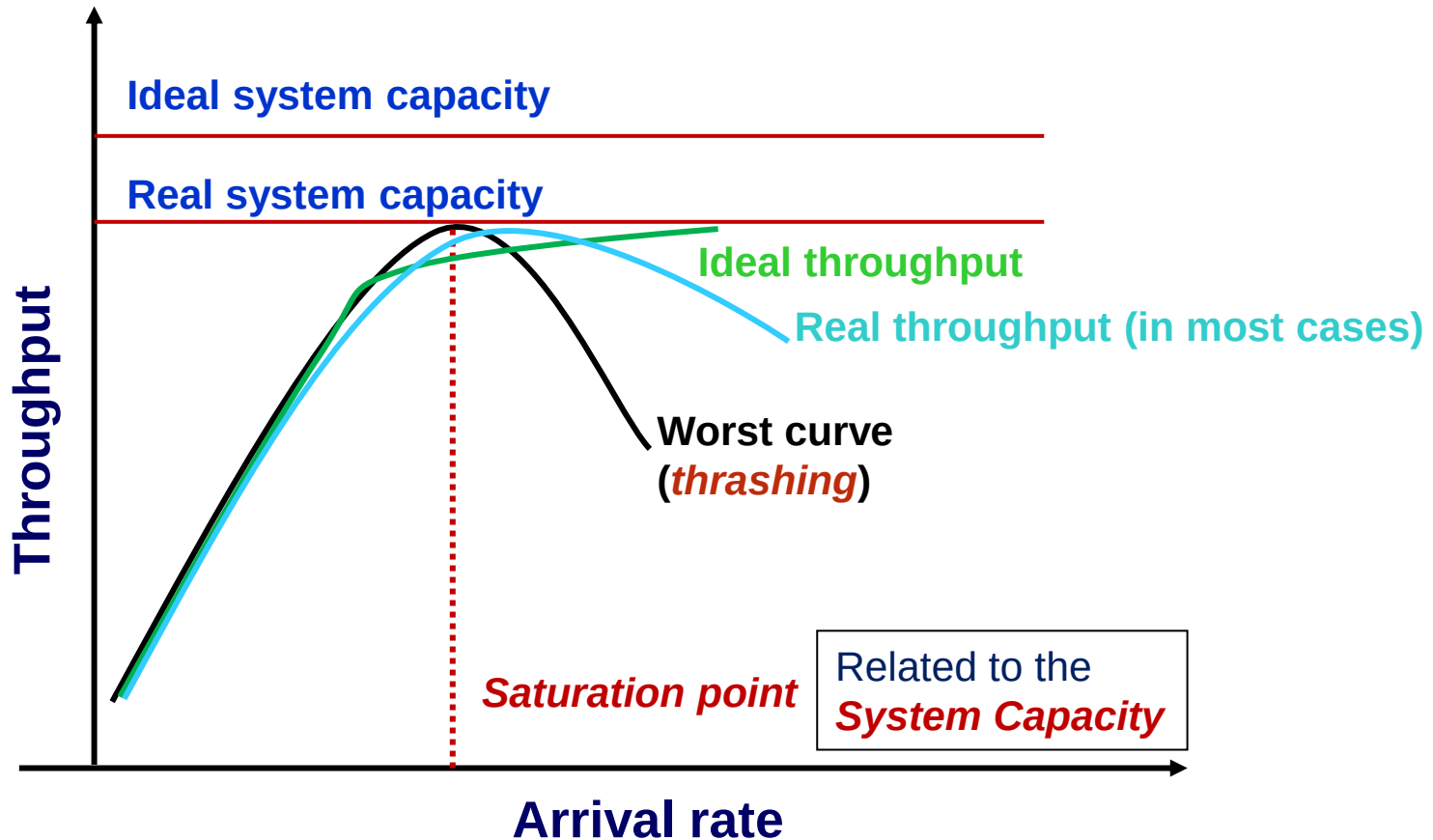
Examples

- Number of processed requests per minute
- Number of customers served per day
- Number of I/O transactions per hours
- Number of transmitted packets per minute
- Number of instructions per second

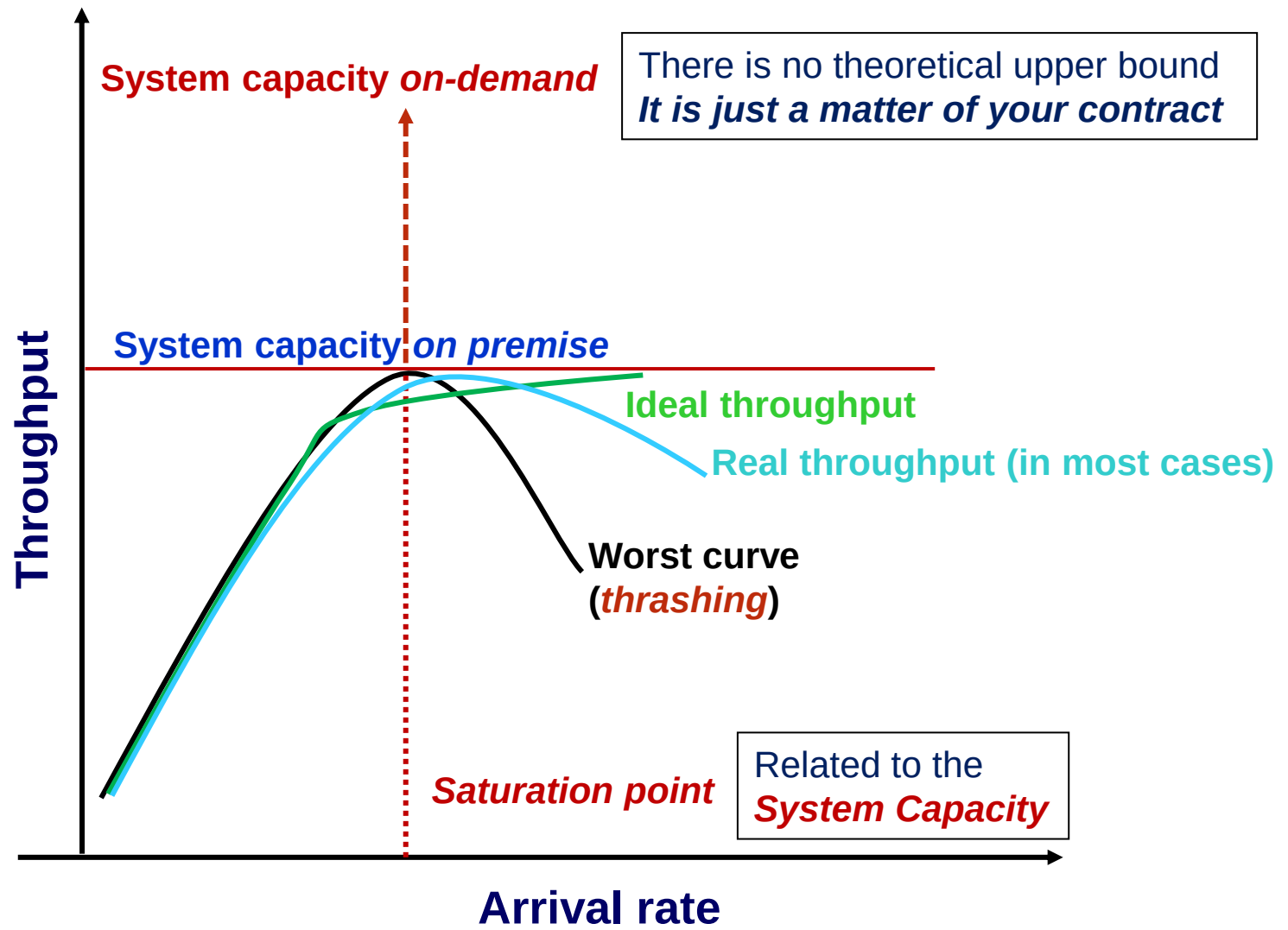
– ...



Throughput as a function of the arrivals



The advantage of cloud IaaS



Scalability

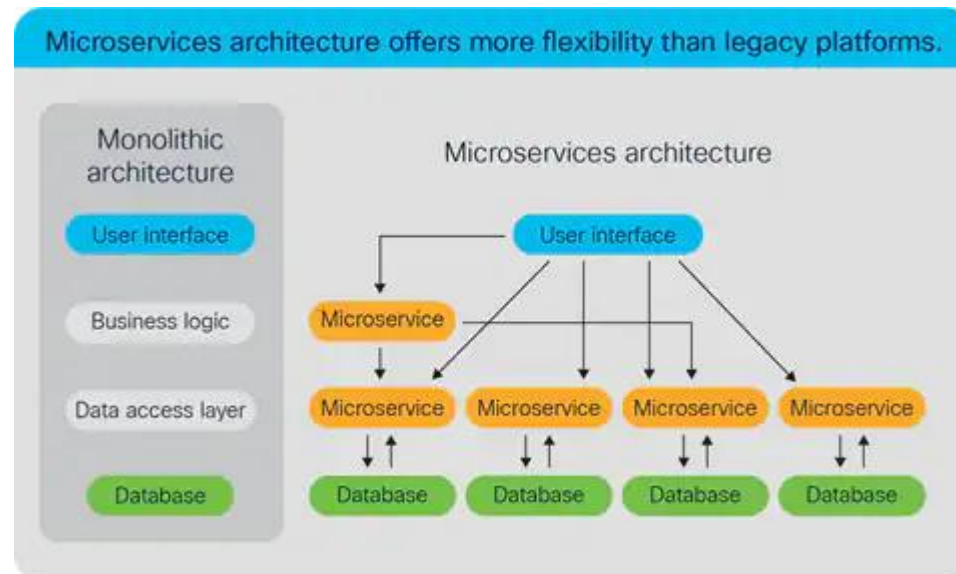
- Many traditional services begin with a **monolithic architecture**
 - Programming is easier because is focused on one target
 - It works fine for low traffic and when applications are characterized by a stable workload (that represents the most common starting point)
- When the workload increases or gets more variable, the performance can degrade and at some point, the service might crash if scalability is not guaranteed



Modern solution

To design from the beginning architectures and applications that can scale “infinitely” requires:

- **Microservices**
- **Fast provisioning of functions**
- **Autonomous load distribution**



Scalable and reliable architectures

AVOID

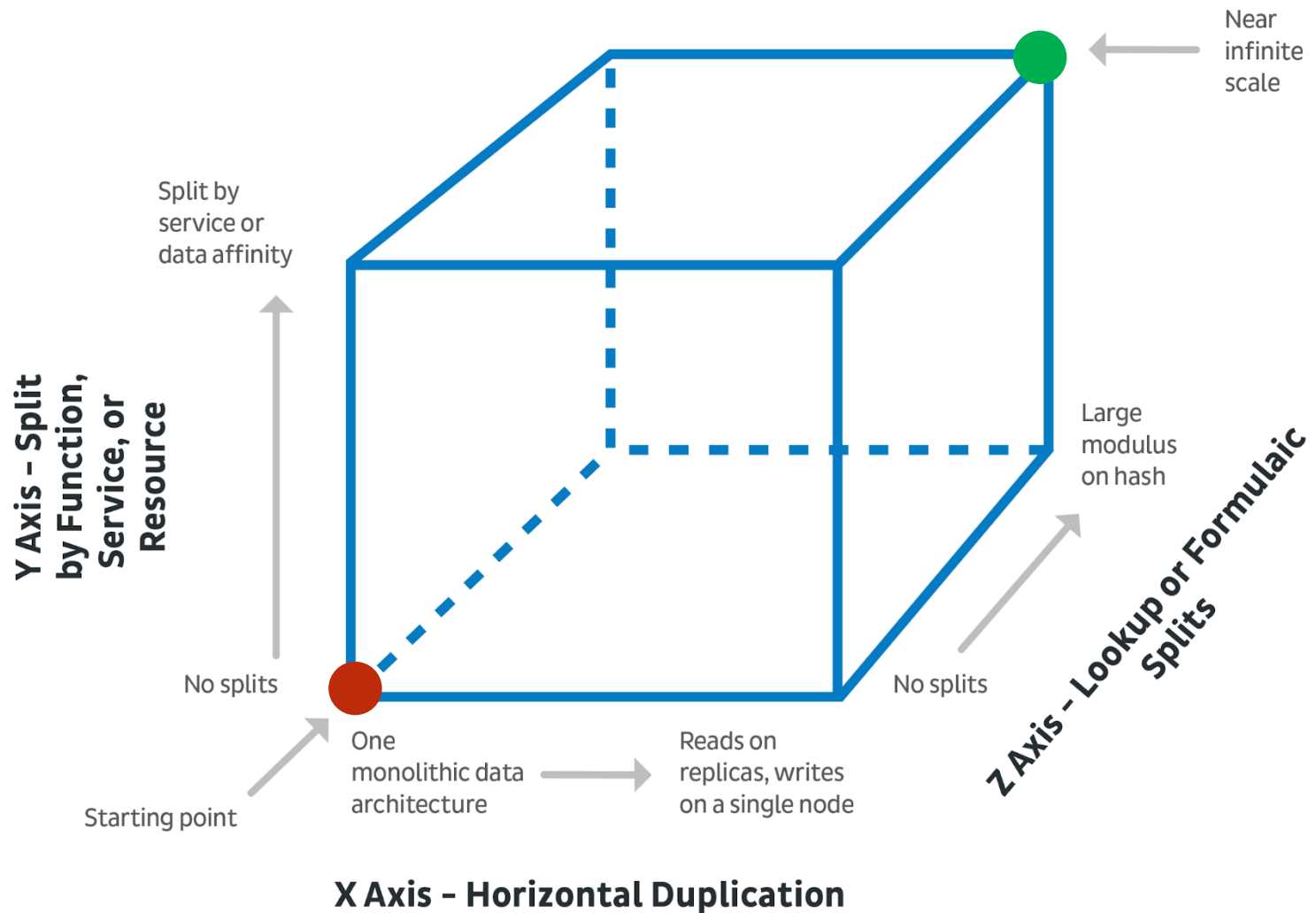
- **System bottlenecks**
- **Single points of failure**

through

- **Replication at different levels
(processes and data)**
- **Fault isolation**



Possible *design style*: The XYZ scale cube



X axis scaling

X is the dimension of *replication of services and data*

- The goal is to run **N** instances of replicated applications and/or replicated databases
- It is necessary to introduce a suitable **load balancer** with the goal that each instance can approximately handle **1/N** of the workload (the *Kubernetes* control plane automates the creation of the external *load balancer*, health checks, and packet filtering rules)

PROS

- Easy to design and implement
- Scales well

CONS

- Higher costs
- Does not address caching issues
- Does not address organizational scalability issues



X axis scaling

- Running multiple identical copies of the application behind a load balancer is a great way of improving the capacity and the availability of an application
- One benefit of the X axis is that it is typically intellectually easy to implement, and it scales well from a transaction perspective
- Impediments to implementing the X axis include heavy session related information which is often **difficult to distribute** or requires **persistence** to servers (they can cause reliability and scalability problems)
- Drawbacks to the X axis is that, while easy to implement, data sets have to be replicated entirely and this increases operational costs. Further, caching tends to degrade at many levels as the size of data increases with transaction volumes. Finally, the X axis does not engender higher levels of organizational scale

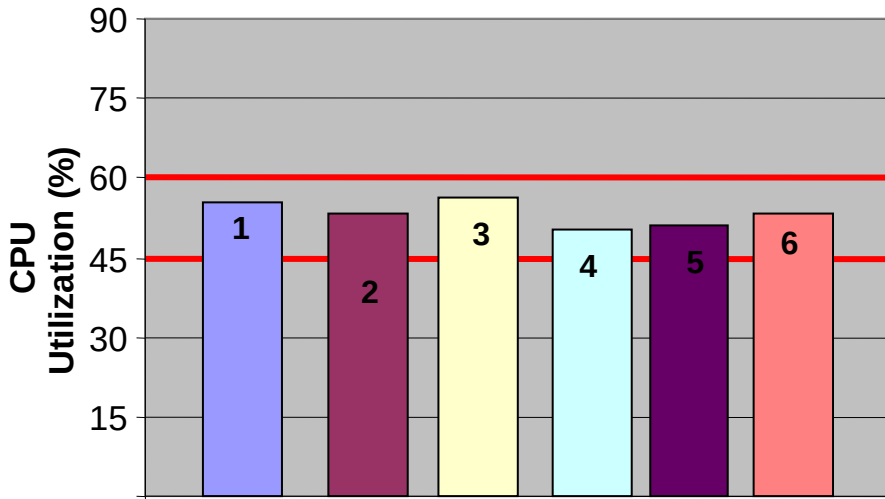


Example

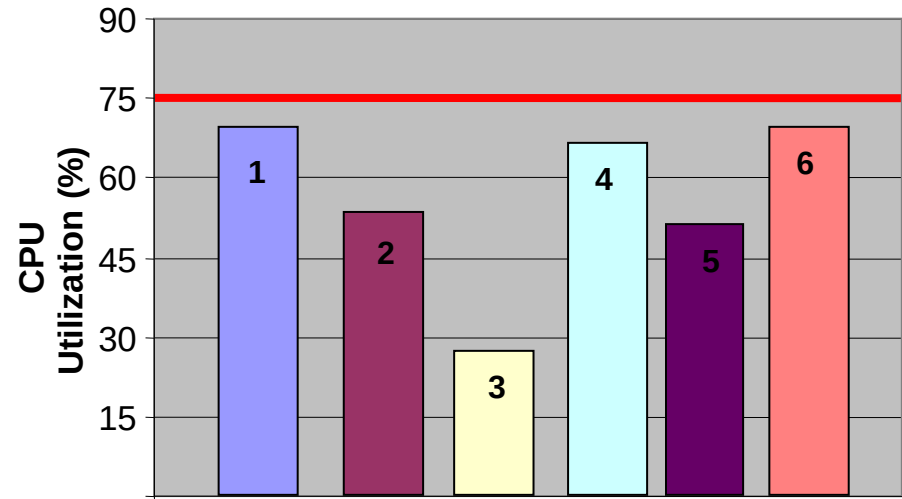
- **Presentation tier:** replicate *http servers* and use a load balancer
- **Application tier:** Store Session in browser or separated Objects. Cache to horizontally scale application tier independent of presentation tier
- **Database tier:** Use read-replicas for read-only operations like reporting, summary, search, etc.



Load balancing vs Load sharing



***Load balancing* →
“Equalize the load”**



***Load sharing* →
“Avoid critical load cases”**

***Load sharing* makes the scheduling problem tractable,
but it does not make the problem naive**



Y axis scaling

Y is the dimension of *functional decomposition*

- Monolithic applications are separated along functional or resource-oriented boundaries to create macro and microservices
- The idea is to scale each service independently and to assign more resources only to the services that need them

PROS

- Scales transactions well
- Allows fault isolation
- Allows organization scalability

CONS

- It is difficult to design
- It takes time to implement



Example

- Y-axis scaling (services oriented architecture, microservices or functional decomposition of a monolith) focuses on separating services and data along noun or verb boundaries. These splits are “dissimilar” from each other
- Examples in commerce solutions may be:
 - splitting search from browse
 - checkout from add-to-cart
 - login from account status
 - etc.



Implementation

Y axis scaling shares the benefit of increasing transaction scalability with all the axes of the cube. In implementing splits, Y-axis scaling splits a monolithic application into a set of (micro-)services

- Each service implements a set of related functions such as order management, customer management, inventory, etc.
- Each service should have its own, non-shared data to ensure high availability and fault isolation
- Further, because the Y axis allows segmentation of teams and ownership of code and data, organizational scalability is increased
- Cache hit ratios should increase as data and the services are appropriately segmented and similarly sized memory spaces can be allocated to smaller data sets accessed by comparatively fewer transactions
- Operational cost often is reduced as systems can be sized down to commodity servers or smaller IaaS instances can be employed



Z axis scaling

Z is the dimension of *splitting* service and data

Each server may run the same code but for a subset of data

- A common implementation for B2C services is geographically splitting of customers into *pods*. This solution can also diminish network latencies and achieve faster response time
- For B2B services, Z axis split is done in pods along the boundaries of a company or of a group of companies

PROS

- Easy to design
- Scales transactions well and can improve response time
- Can provide fault isolation

CONS

- Implementation takes time and reducing overhead requires automation
- Does not address organizational scale



Z axis scaling

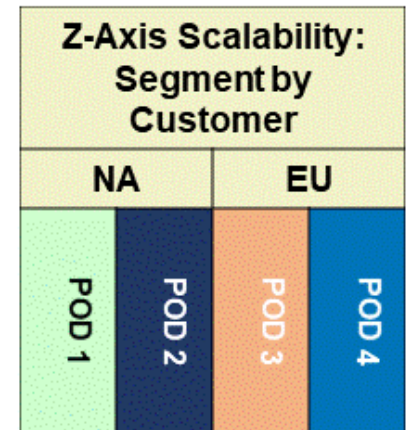
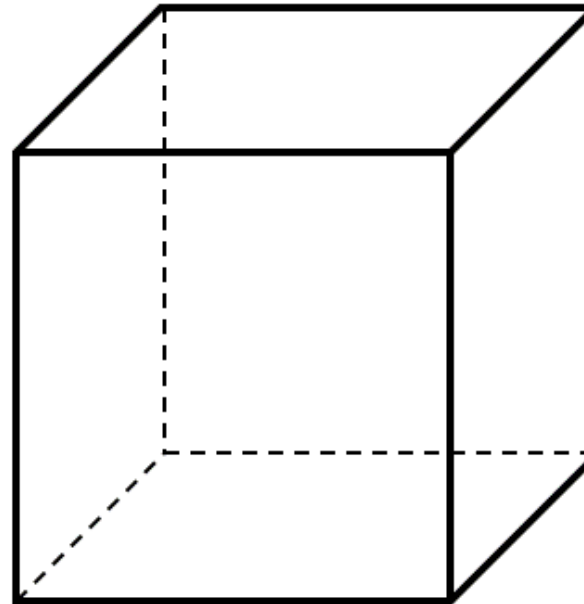
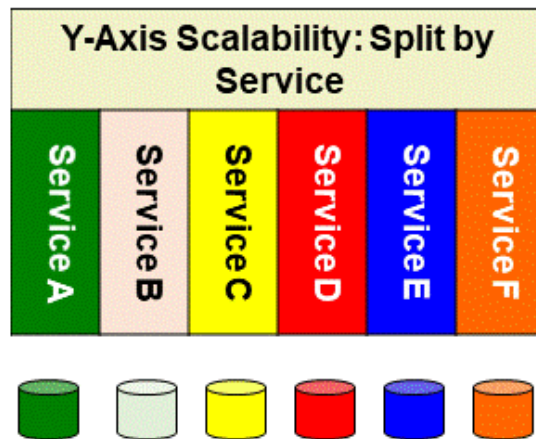
Whereas the Y axis addresses the splitting of dissimilar things, the Z-axis addresses segmentation of “similar” things

Examples:

- splitting customers along an unbiased modulus of customer_id, or along a somewhat biased (but beneficial for response time) geographic boundary
- product catalogs may be split by Stock Keeping Unit
- content may be split by content_id
- Z-axis scaling, like all of the axes, improves the solution’s transactional scalability and, *if faults are isolated*, its availability
- Because the software deployed to servers is essentially the same in each Z axis shard (but the data is distinct) there is no increase in organizational scalability needs
- Cache hit rates often go up with smaller data sets
- Operational costs generally go down as commodity servers or smaller IaaS instances can be used



Summary: AKF scale cube



X-Axis Scalability: Replicate & LB	
Web Tier	Replicate Web Servers & Load Balance
App Tier	Store Session in browser or separated Object Cache to horizontally scale app tier independent of web tier
DB Tier	Use Read-Replicas for read-only use cases like reporting, search, etc.



The importance of data replication

- Replication at different levels (**processes** and **data**)
- But we know that *data consistency* represents one of the most difficult issue for scalability

Which type(s) of data can we replicate?

NEXT LESSONS →

